

Julia Programming Language

Julia combines high-level expressive syntax with high-performance execution, making it particularly well-suited for scientific computing, data science, numerical analysis, and systems-level research workflows.

***created by Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman

Print

print to screen:

```
println("Hello, world!")
```

Markdown

markdown: used in julia to write text in Pluto notebooks by use
'md'''' string inside Pluto cell

```
md" "
```

```
This text is in bold and italic.
```

```
""
```

keyboard shortcut:

Ctrl+M

Image in
Markdown

method 1: access online image

target url: <https://i.imgur.com/qHCI8RS.png>

```
md" "
```

```
![Description of the  
image] (https://i.imgur.com/qHCI8RS.png)
```

```
""
```

method 2: local image

for a local image to render in the notebook, the image has to
be packaged and stored in the same or child directory with the
notebook

target image: ./image.png

*Control Flow***if statements: conditional evaluation**

```
using PlutoUI

PlutoUI.LocalResource("./images/image.png")
```

```
if x < y
    println("x is smaller")
else
    println("y is smaller")
end
```

for loop: range-restricted iteration

```
for i in 1:5
    println(i)
end
```

while loop: conditional iteration

```
n = 0
while n < 3
    n += 1
end
```

*Data Structures***arrays:**

```
arr = [1, 2, 3]
```

```
push!(arr, 4)
```

push!

inserts items into a mutable collection by changing the existing collection instead of creating a new one

dict:

```
phonebook = Dict{"Alice"=>"1234", "Bob"=>"5678"}
```

Collection
Manipulation

type definition:

```
struct Point
    x::Float64
    y::Float64
end
```

list comprehensions

```
squares = [i^2 for i in 1:10]
```

higher-order function

```
apply_twice(f, x) = f(f(x))
```

map / filter

```
evens = filter(x -> x % 2 == 0, 1:10)
```

Multiple Dispatch

core Julia design feature: the specific implementation of a function is chosen at runtime based on the data types of all of its arguments

```
f(x::Int) = "integer"
f(x::String) = "string"
```

this allows generic functions to have many specialized behaviors for different combinations of input types

method on type: defines a function using ‘::’ notation

```
distance(a::Point, b::Point) =
    sqrt((a.x-b.x)^2 + (a.y-b.y)^2)
```

Euclidean Distance
Formula in Julia

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

generic function with dispatch

Julia chooses
which
method to
implement
based on
number of
arguments

*Tuple Unpacking***comma delineated dual assignment**

```
area(x::Number, y::Number) = x*y
area(x::AbstractVector) = sum(x)
```



```
a, b = 1, 2
```

equates to,

```
a = 1, b = 2
```

*Interactivity UI**Elements:**Sliders***@bing with PlutoUI****create interactive UI elements**

```
using PlutoUI

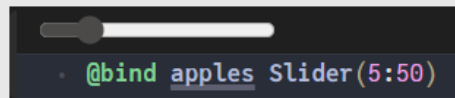
@bind apples Slider(5:50)

apples

repeat("🍏", apples)
```

output:

the @bind command creates the slider with a range(5:50)



adjusting the slider adjust the number of apples displayed

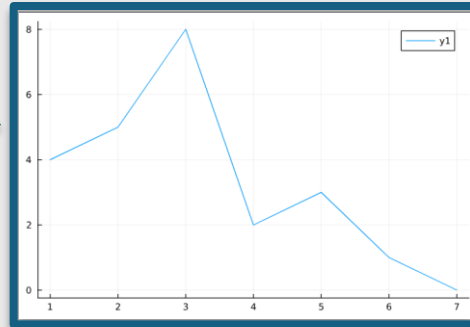
“🍏🍏🍏🍏🍏🍏🍏🍏”

*Plots***there are several plotting libraries available****examples:** Makie.jl, PlutoPlotly.jl, Gadify.jl, Plots.jl**Plots.jl**

```
using Plots

plot([4,5,8,2,3,1,0])
```

result



combining multiple plot calls in a single cell

```
data = rand(100)::Vector{Float64}
```

```
let
```

```
plot(data)
```

```
# use the ! exclamation mark to modify the previous plot
```

```
scatter!(data)
```

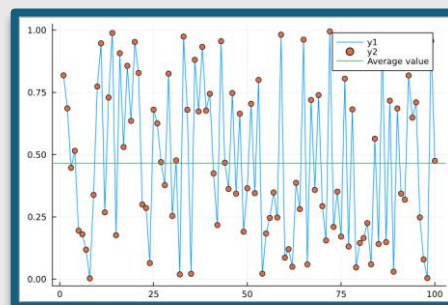
```
# add a horizontal line
```

```
mean = sum(data) / length(data)
```

```
hline!([mean]; label="Average value")
```

```
end
```

result



A `let` block creates a **local scope**, allowing you to use mutable operations like `plot!` and `hline!` to **modify an existing plot** to build a single complex object.

LaTeX Pluto has built-in LaTeX support for math equations for use inside markdown

method 1: inline math

```
md"""
```

```
inline equation: ``\frac{1}{2 \pi}``.
```

```
"""
```

notice double
back-tics

note: the legacy

LaTeX syntax:

`$\frac{1}{2 \pi}$`

is deprecated

.

Links

the '#' character can be used to link to specific sections in a Pluto notebook:

global variables:	<code>#variable_name</code>
function definitions:	<code>#function_name</code>
markdown headers:	<code>#header-text</code>

inside the notebook:

```
md"""
Take a look at [the fruits variable] (#fruits).
"""
```

displayed text link name

The diagram shows two methods for rendering mathematical expressions in a Pluto notebook. At the top, an **inline equation** is shown as $\frac{1}{2\pi}$, with an arrow pointing to it labeled **inline result**. Below this, **method 2: block math** is demonstrated. It shows the code: `md"""`, `block equation:`, ````math`, `\frac{1}{2 \pi}`, `````, and `"""`. An arrow points from the code to a **block result** box containing the rendered equation $\frac{1}{2\pi}$.

<i>References</i>	Julia. (n.d.) Tutorials Point https://www.tutorialspoint.com/julia/julia_tutorial.pdf <i>Julia Documentation · The Julia Language.</i> (n.d.). https://docs.julialang.org/en/v1/ <i>Linking.</i> (n.d.). https://plutojl.org/en/docs/linking/ <i>Markdown.</i> (n.d.). https://plutojl.org/en/docs/markdown/ Wikipedia contributors. (2026, February 11). <i>Julia (programming language)</i>. Wikipedia. https://en.wikipedia.org/wiki/Julia_(programming_language)
-------------------	---